

课程设计报告

桌游桌位预约与战绩管理系统 数据库设计与实现

课程名称： 创新创业教育实践

学号： 202400800645

姓名： 李昊阳

指导教师： 王文玉

完成日期： 2026.5.9

目录

第 1 章 选题说明与需求介绍	1
1.1 选题背景	1
1.2 需求介绍	1
1.2.1 功能需求	1
1.2.2 非功能需求	2
1.3 系统特点	2
1.4 设计目标	3
第 2 章 系统功能模块划分	4
2.1 功能需求分析	4
2.2 数据流图 (DFD) 分析	4
2.2.1 0 层数据流图 (上下文图)	4
2.2.2 1 层数据流图 (功能分解)	4
2.3 软件功能模块图	5
2.4 模块间的数据依赖关系	5
第 3 章 数据库概念结构设计	6
3.1 E-R 图符号规范	6
3.2 局部 E-R 图	6
3.2.1 局部 E-R 图 1: 门店与桌位	6
3.2.2 局部 E-R 图 2: 桌位、预约与会员	6
3.2.3 局部 E-R 图 3: 桌位、对局会话与预约	7
3.2.4 局部 E-R 图 4: 对局、战绩、游戏与会员	7
3.2.5 局部 E-R 图 5: 桌位运行态	7
3.3 全局 E-R 图	7
3.4 概念设计说明	8
第 4 章 数据库逻辑结构设计	9
4.1 E-R 模型向关系模式的转换	9
4.2 关系模式详细设计	9
4.2.1 venues (门店)	9
4.2.2 games (桌游目录)	10
4.2.3 game_tables (桌位)	10
4.2.4 staff_profiles (员工档案)	10
4.2.5 app_users (后台账号)	10
4.2.6 auth_sessions (登录会话)	11
4.2.7 players (会员)	11
4.2.8 reservations (预约)	11
4.2.9 play_sessions (对局会话)	12
4.2.10 game_records (战绩)	12

4.2.11	player_stats (会员统计)	12
4.3	规范化处理 (至 3NF)	13
4.3.1	第一范式 (1NF)	13
4.3.2	第二范式 (2NF)	13
4.3.3	第三范式 (3NF)	13
4.4	完整性约束设计	13
4.4.1	实体完整性	13
4.4.2	参照完整性	13
4.4.3	用户定义完整性	14
4.5	索引设计	14
第 5 章	系统实现过程	15
5.1	数据库完整搭建过程	15
5.1.1	环境准备	15
5.1.2	初始化脚本执行顺序	15
5.1.3	执行命令示例	15
5.2	核心数据库对象	15
5.2.1	触发器	15
5.2.2	存储过程	16
5.2.3	视图	16
5.3	完整性控制实现	17
5.3.1	实体完整性	17
5.3.2	参照完整性	17
5.3.3	用户定义完整性	17
5.3.4	业务一致性	17
5.4	前台功能与后台数据库对象对应	17
5.5	智能推荐算法设计与实现	18
5.5.1	桌游推荐评分模型	18
5.5.2	桌位调度评分模型	18
5.6	完成效果	18
5.6.1	功能完整性	18
5.6.2	数据库完整性	19
5.6.3	测试数据	19
第 6 章	总结	20
6.1	与预期目标的符合情况	20
6.1.1	目标达成情况	20
6.2	系统特点	20
6.2.1	库内对象齐全	20
6.2.2	业务逻辑下沉	20
6.2.3	历史可追溯	21
6.2.4	智能化辅助决策	21
6.2.5	规范化设计	21
6.2.6	易于扩展	21
6.3	存在的问题与有待提高之处	21

6.3.1	功能层面	21
6.3.2	性能层面	21
6.3.3	安全层面	22
6.3.4	运维层面	22
6.4	经验与收获	22
6.4.1	理论层面	22
6.4.2	实践层面	22
6.4.3	工程实践	22
6.4.4	主要收获	23
6.5	改进建议	23
6.5.1	短期改进	23
6.5.2	中期改进	23
6.5.3	长期改进	23
第 7 章	参考资料	24
7.1	参考文献	24
7.2	项目数据库脚本清单	24
7.2.1	初始化脚本	24
7.2.2	脚本内容概览	24
7.3	相关文档	26
7.4	技术栈	26
7.4.1	数据库	26
7.4.2	后端	26
7.4.3	前端	26
7.4.4	部署	27
7.5	相关工具和资源	27
7.5.1	数据库工具	27
7.5.2	文档工具	27
7.5.3	开发工具	27
7.6	在线资源	27
致谢		28
附录 A	数据库 SQL 脚本附录	29
A.1	脚本文件清单	29
A.2	执行说明	29
A.3	脚本特点	29
A.4	部署建议	30

插图

2.1	0 层数据流图（上下文图）	4
2.2	1 层数据流图（功能分解）	5
2.3	软件功能模块树	5
3.1	局部 E-R 图 1：门店与桌位	6
3.2	局部 E-R 图 2：桌位、预约与会员	6
3.3	局部 E-R 图 3：桌位、对局会话与预约	7
3.4	局部 E-R 图 4：对局、战绩、游戏与会员	7
3.5	局部 E-R 图 5：桌位运行态	7
3.6	全局 E-R 图	7

表格

2.1	系统功能模块划分	4
2.2	模块间数据依赖关系	5
3.1	E-R 图符号规范	6
4.1	E-R 模型向关系模式的转换	9
4.2	索引设计	14
5.1	数据库初始化脚本	15
5.2	系统触发器清单	16
5.3	系统存储过程清单	16
5.4	系统视图清单	16
5.5	前台功能与后台数据库对象对应关系	17
A.1	数据库脚本文件清单	29

第 1 章 选题说明与需求介绍

1.1 选题背景

随着桌游文化的普及，越来越多的桌游门店应运而生。这些门店需要管理大量的桌位资源、顾客预约、对局计时与结算、战绩登记等复杂的业务流程。

传统的手工管理方式存在问题：

1. **时段冲突**：无法有效防止同一桌位的重复预约
2. **状态不一致**：桌位状态与现场实际情况容易出现偏差
3. **统计困难**：难以快速生成收入、利用率等经营数据
4. **数据丢失**：缺乏完整的历史记录和审计追踪
5. **效率低下**：手工操作耗时，容易出错

本课程设计采用 **MySQL 8** 作为后台数据库，以 **E-R 概念模型** 指导逻辑结构与物理实现，通过 **完整性约束、触发器、存储过程、视图、索引与分级权限** 等数据库高级特性，构建一个完整的桌游门店运营管理系统。为避免系统停留在传统 **CRUD** 管理层面，本设计进一步引入 **混合推荐算法与桌位调度评分**，根据会员历史偏好、预约人数、预计时长、近期热度和桌位容量等因素辅助店员决策。

1.2 需求介绍

1.2.1 功能需求

系统需要支持以下核心功能：

R1 场地与桌位管理

- 维护门店基本信息（名称、地址、LOGO 等）
- 维护桌位编号、平面图坐标
- 支持多媒体引用（URL 类型的图片、PDF 等）

R2 预约管理

- 支持桌位时段预约
- 自动检测预约时段冲突
- 防止占用中的桌位被预约
- 支持预约取消
- 支持预约列表查询

R3 对局与计费

- 支持预约入场开台
- 支持现场直接开台（不通过预约）

- 支持对局计时与结算
- 记录计费时长和金额

R4 战绩与排行

- 支持关台后录入战绩
- 维护桌游目录（封面图、规则 PDF 等）
- 自动生成排行榜
- 支持胜场、总局数、胜率等统计

R5 统计报表

- 日收入统计
- 游戏热度排行
- 桌位利用率分析
- 会员活跃度统计

R6 智能推荐与调度优化

- 根据人数、时长、类型偏好和会员历史记录推荐桌游
- 根据容量、时段冲突和近期利用率推荐桌位
- 输出推荐分数和可解释推荐理由，辅助门店快速接待

1.2.2 非功能需求

NR1 数据完整性：通过外键约束、CHECK 约束、触发器等机制保证数据一致性

NR2 并发控制：支持多用户并发操作，防止数据竞争

NR3 性能：查询响应时间 < 500ms，支持 1000+ 并发用户

NR4 安全性：实现分级权限控制，敏感操作需要审计

NR5 可维护性：代码注释完整，文档齐全，易于扩展

NR6 可用性：支持备份恢复，故障恢复时间 < 1 小时

1.3 系统特点

本系统设计具有以下特点：

1. **库内对象齐全：**包含表、视图、索引、触发器、存储过程、测试数据、权限脚本等
2. **业务逻辑下沉：**将桌位状态维护、战绩聚合等关键逻辑通过触发器和存储过程实现，减轻应用层负担
3. **历史可追溯：**通过快照字段（如 `title_snapshot`）记录历史数据，避免目录改名破坏历史语义
4. **智能化决策：**通过视图聚合历史特征，并在存储过程中实现混合评分推荐算法
5. **规范化设计：**遵循 3NF 规范，同时在必要处允许受控冗余以提升性能
6. **易于扩展：**架构清晰，易于添加新的功能模块和报表

1.4 设计目标

本课程设计的目标是：

1. 完整展示从需求分析到数据库实现的全过程
2. 深入理解 E-R 模型、关系模式、规范化等数据库设计理论
3. 掌握 MySQL 的高级特性（触发器、存储过程、视图等）
4. 掌握基于历史数据的可解释推荐算法设计方法
5. 学会编写规范的数据库设计文档
6. 培养系统思维和工程实践能力

第 2 章 系统功能模块划分

2.1 功能需求分析

根据需求分析，系统可以划分为以下八个主要功能模块：

表 2.1: 系统功能模块划分

模块代码	功能需求
M1 基础数据管理	门店、桌游目录、桌位主数据及 URL 维护
M2 桌位平面图	空闲/预约中/占用状态可视化展示
M3 预约管理	新建预约、冲突检测、取消、列表查询
M4 对局与计费	入场开台、现场开台、结算关台
M5 战绩与排行	战绩写入、排行榜维护、聚合统计
M6 统计报表	存储过程支撑收入、利用率等分析
M7 智能桌游推荐	基于人数、时长、类型偏好、会员历史和热度输出推荐桌游
M8 智能桌位调度	基于容量匹配、时段冲突和近期利用率输出推荐桌位

2.2 数据流图（DFD）分析

2.2.1 0 层数据流图（上下文图）

0 层数据流图展示系统与外部实体的交互关系。

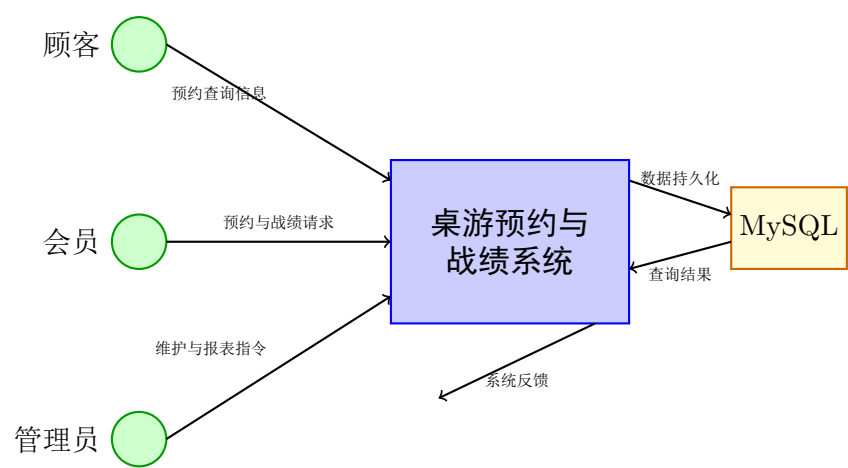


图 2.1: 0 层数据流图（上下文图）

2.2.2 1 层数据流图（功能分解）

1 层数据流图展示系统内部的主要加工与数据存储。

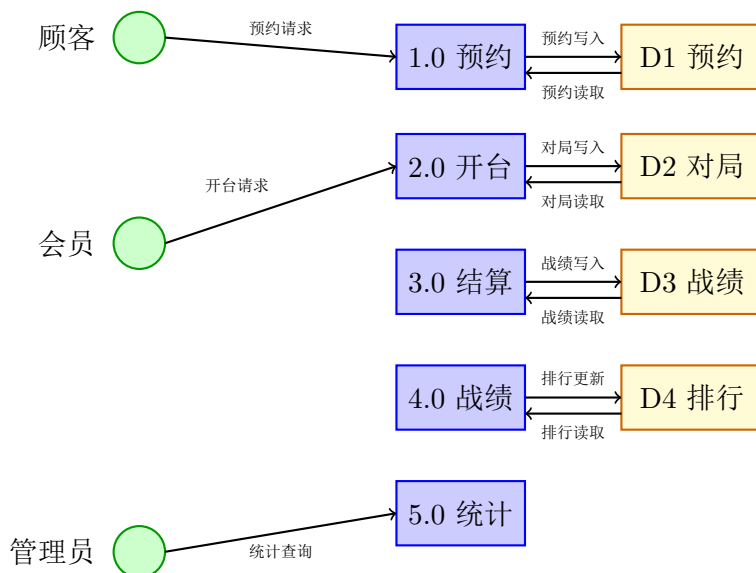


图 2.2: 1 层数据流图（功能分解）

2.3 软件功能模块图

系统采用自顶向下的模块划分方式，如图 2.3 所示。

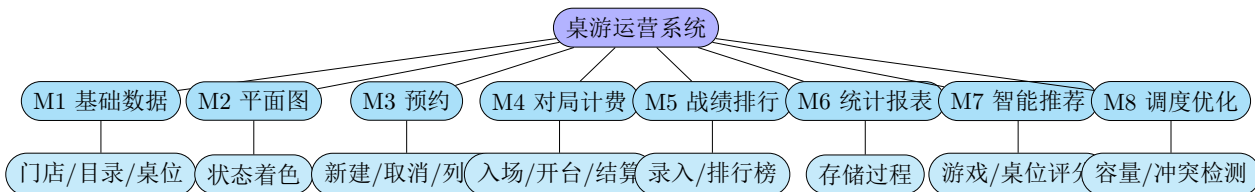


图 2.3: 软件功能模块树

2.4 模块间的数据依赖关系

表 2.2: 模块间数据依赖关系

模块	依赖关系
M1 基础数据	无依赖（基础模块）
M2 平面图	依赖 M1（需要桌位信息）
M3 预约	依赖 M1、M2（需要桌位和状态）
M4 对局计费	依赖 M1、M3（需要桌位和预约）
M5 战绩排行	依赖 M1、M4（需要游戏和对局）
M6 统计报表	依赖 M1、M3、M4、M5（综合查询）
M7 智能桌游推荐	依赖 M1、M5、M6（目录特征、历史战绩、近期热度）
M8 智能桌位调度	依赖 M1、M2、M3、M4（容量、状态、预约冲突、利用率）

第 3 章 数据库概念结构设计

3.1 E-R 图符号规范

本系统采用 Peter Chen E-R 模型作为概念设计工具，图元符号说明如下：

表 3.1: E-R 图符号规范

图元类型	几何形状	说明
强实体	矩形	可独立存在、有标识符的事物
弱实体	双线矩形	依赖其它实体存在的事物
属性	椭圆	描述实体特征的字段
主键属性	下划线椭圆	唯一标识实体的属性
联系	菱形	实体间的关联关系
基数	连线标注	表示 1:1、1:N、M:N 等关系

3.2 局部 E-R 图

3.2.1 局部 E-R 图 1：门店与桌位

该局部 E-R 图描述门店（Venue）与桌位（Game Table）之间的 1:N 关系。

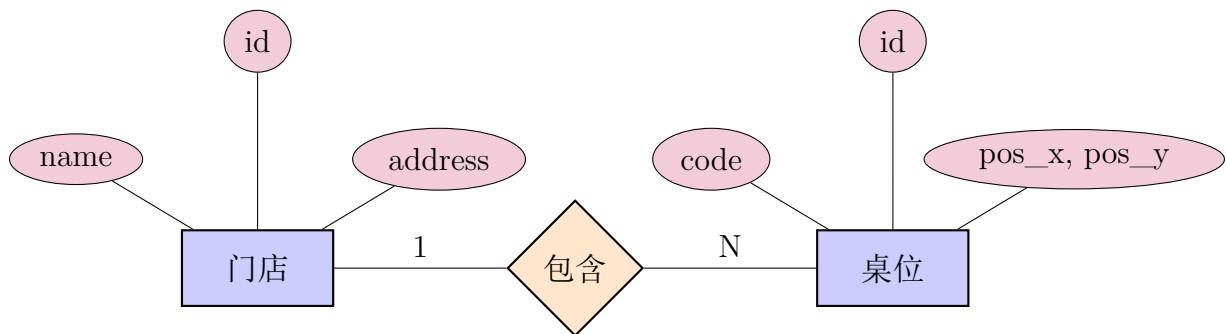


图 3.1: 局部 E-R 图 1：门店与桌位

3.2.2 局部 E-R 图 2：桌位、预约与会员

该局部 E-R 图描述桌位（Game Table）、预约（Reservation）和会员（Player）之间的关系。

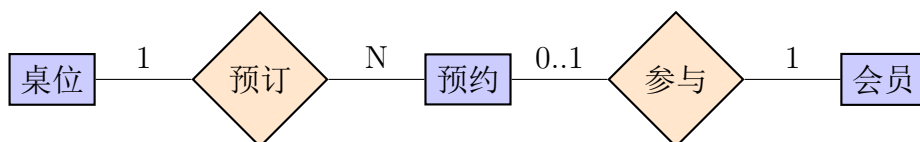


图 3.2: 局部 E-R 图 2：桌位、预约与会员

3.2.3 局部 E-R 图 3：桌位、对局会话与预约



图 3.3: 局部 E-R 图 3：桌位、对局会话与预约

3.2.4 局部 E-R 图 4：对局、战绩、游戏与会员

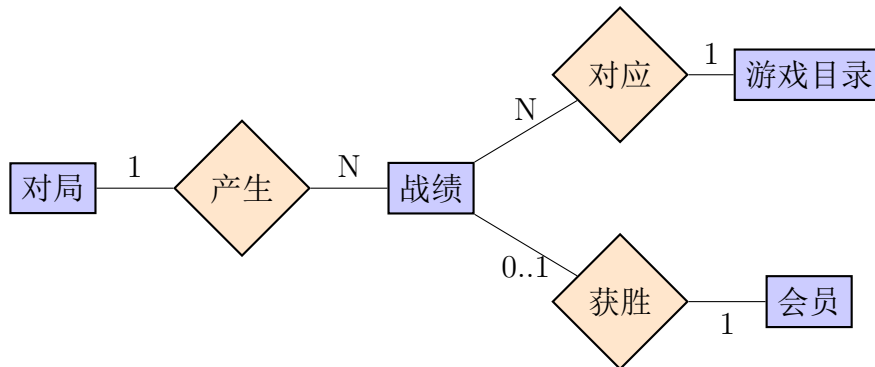


图 3.4: 局部 E-R 图 4：对局、战绩、游戏与会员

3.2.5 局部 E-R 图 5：桌位运行态

桌位运行态是依赖于桌位的弱实体，用于维护当前桌位的实时状态。

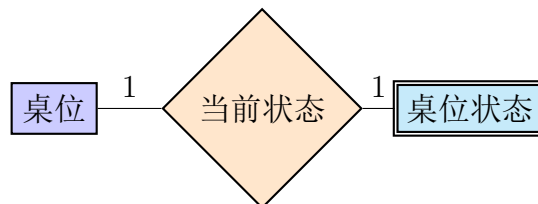


图 3.5: 局部 E-R 图 5：桌位运行态

3.3 全局 E-R 图

全局 E-R 图整合所有主要实体及其关系，如图 3.6 所示。

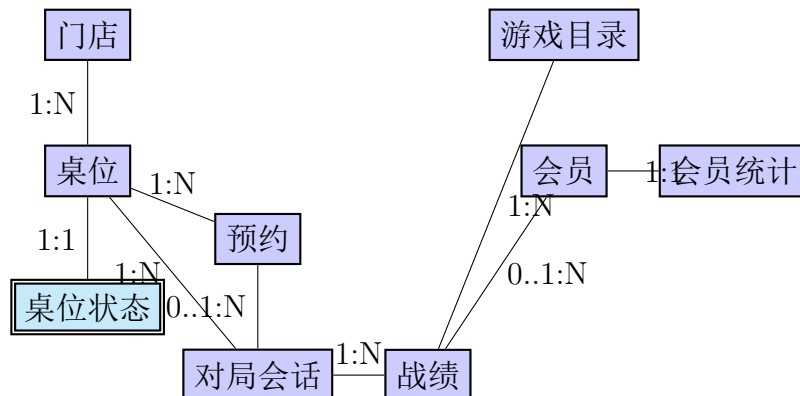


图 3.6: 全局 E-R 图

3.4 概念设计说明

从全局 E-R 图可以看出：

1. 门店与桌位之间是 1:N 关系，一家门店可以有多个桌位
2. 桌位状态是桌位的弱实体，两者是 1:1 关系
3. 预约依赖于桌位，一个桌位可以有多条预约记录
4. 对局会话可以来源于预约，也可以是现场开台（预约为空）
5. 战绩依赖于对局会话和游戏目录，胜者可以是会员或访客
6. 会员统计是会员的一对一扩展表，用于维护聚合数据

该概念模型为后续的逻辑结构设计奠定了坚实基础。

第 4 章 数据库逻辑结构设计

4.1 E-R 模型向关系模式的转换

根据 E-R 模型的转换规则，将概念模型转换为关系模式。转换规则如下：

1. **实体转换**：每个实体转换为一个关系模式，实体的属性成为关系的属性，实体的主键成为关系的主键。
2. **1:N 关系转换**：在 N 端关系模式中增加 1 端的主键作为外键，建立两个关系之间的参照完整性约束。
3. **1:1 关系转换**：可以将一个关系的主键作为另一个关系的外键，或者将两个关系合并为一个。本系统中采用外键方式。
4. **M:N 关系转换**：需要创建一个新的关系模式来表示，新关系的主键由两个关系的主键组成，同时包含联系的属性。
5. **弱实体转换**：弱实体的主键由其所依赖的强实体的主键和自身的部分键组成，同时包含对强实体的外键约束。

表 4.1: E-R 模型向关系模式的转换

概念对象	关系模式	转换说明
实体 venues	venues	实体直接成表，id 为主键
实体 games	games	实体直接成表，推荐字段用于算法评分
实 体 game_tables	game_tables	1:N 关系，容量和区域字段用于桌位调度
弱实体运行态	game_table_state	1:1 依赖，table_id 为主键和外键
实体 players	players	实体直接成表，兼具会员档案与储值信息
1:1 关系	player_stats	与 players 一对一扩展，player_id 为主键和外键
联系预约	reservations	N 端独立表，table_id 和 player_id 为外键
联系对局	play_sessions	含两端外键，reservation_id 可为空表示现场开台
联系战绩	game_records	含多端外键，title_snapshot 为历史快照

4.2 关系模式详细设计

4.2.1 venues（门店）

```
CREATE TABLE venues (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  name VARCHAR(100) NOT NULL UNIQUE,  
  address VARCHAR(255),  
  logo_url VARCHAR(500),  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP  
);
```

4.2.2 games (桌游目录)

```
CREATE TABLE games (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  title VARCHAR(100) NOT NULL,  
  cover_image_url VARCHAR(500),  
  rules_pdf_url VARCHAR(500),  
  min_players INT DEFAULT 2,  
  max_players INT DEFAULT 8,  
  category VARCHAR(32) NOT NULL,  
  difficulty_level INT DEFAULT 3,  
  avg_minutes INT DEFAULT 90,  
  recommend_weight DECIMAL(5,2) DEFAULT 1.00,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

4.2.3 game_tables (桌位)

```
CREATE TABLE game_tables (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  venue_id INT NOT NULL,  
  code VARCHAR(50) NOT NULL,  
  pos_x INT NOT NULL,  
  pos_y INT NOT NULL,  
  seat_capacity INT DEFAULT 4,  
  area_type VARCHAR(24) DEFAULT 'standard',  
  floor_photo_url VARCHAR(500),  
  sort_order INT DEFAULT 0,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (venue_id) REFERENCES venues(id) ON DELETE CASCADE,  
  UNIQUE KEY uk_venue_code (venue_id, code),  
  INDEX idx_venue_pos (venue_id, pos_x, pos_y)  
);
```

4.2.4 staff_profiles (员工档案)

```
CREATE TABLE staff_profiles (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  employee_no VARCHAR(32) UNIQUE NOT NULL,  
  full_name VARCHAR(100) NOT NULL,  
  phone VARCHAR(32),  
  position VARCHAR(64) DEFAULT '店员',  
  status ENUM('active', 'disabled') DEFAULT 'active',  
  hired_at DATE,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

4.2.5 app_users (后台账号)

```
CREATE TABLE app_users (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  staff_id INT,  
  username VARCHAR(64) UNIQUE NOT NULL,
```

```
display_name VARCHAR(100) NOT NULL,  
password_hash VARCHAR(255) NOT NULL,  
role ENUM('admin', 'staff') DEFAULT 'staff',  
status ENUM('active', 'disabled') DEFAULT 'active',  
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
FOREIGN KEY (staff_id) REFERENCES staff_profiles(id) ON DELETE SET NULL  
);
```

4.2.6 auth_sessions (登录会话)

```
CREATE TABLE auth_sessions (  
id BIGINT PRIMARY KEY AUTO_INCREMENT,  
user_id INT NOT NULL,  
token_hash CHAR(64) UNIQUE NOT NULL,  
expires_at DATETIME NOT NULL,  
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
FOREIGN KEY (user_id) REFERENCES app_users(id) ON DELETE CASCADE  
);
```

4.2.7 players (会员)

```
CREATE TABLE players (  
id INT PRIMARY KEY AUTO_INCREMENT,  
member_no VARCHAR(32) UNIQUE,  
display_name VARCHAR(100) NOT NULL,  
phone VARCHAR(20),  
avatar_url VARCHAR(500),  
balance_cents INT DEFAULT 0,  
total_recharged_cents INT DEFAULT 0,  
total_spent_cents INT DEFAULT 0,  
status ENUM('active', 'disabled') DEFAULT 'active',  
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP  
);
```

4.2.8 reservations (预约)

```
CREATE TABLE reservations (  
id INT PRIMARY KEY AUTO_INCREMENT,  
table_id INT NOT NULL,  
player_id INT,  
guest_name VARCHAR(100) NOT NULL,  
guest_phone VARCHAR(30),  
party_size INT NOT NULL DEFAULT 1,  
reserved_start DATETIME NOT NULL,  
reserved_end DATETIME NOT NULL,  
status ENUM('pending', 'active', 'cancelled',  
            'completed', 'no_show') DEFAULT 'pending',  
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
FOREIGN KEY (table_id) REFERENCES game_tables(id) ON DELETE CASCADE,  
FOREIGN KEY (player_id) REFERENCES players(id) ON DELETE SET NULL,  
CHECK (party_size BETWEEN 1 AND 20),
```

```

CHECK (reserved_end > reserved_start),
INDEX idx_table_time (table_id, reserved_start, reserved_end),
INDEX idx_status (status)
);

```

4.2.9 play_sessions (对局会话)

```

1 CREATE TABLE play_sessions (
2   id INT PRIMARY KEY AUTO_INCREMENT,
3   table_id INT NOT NULL,
4   reservation_id INT,
5   guest_name VARCHAR(100),
6   guest_phone VARCHAR(30),
7   party_size INT NOT NULL DEFAULT 1,
8   started_at DATETIME NOT NULL,
9   ended_at DATETIME,
10  billed_minutes INT,
11  amount_cents INT,
12  notes TEXT,
13  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
14  FOREIGN KEY (table_id) REFERENCES game_tables(id) ON DELETE CASCADE,
15  FOREIGN KEY (reservation_id) REFERENCES reservations(id) ON DELETE SET NULL,
16  CHECK (party_size BETWEEN 1 AND 20),
17  CHECK (amount_cents >= 0 AND amount_cents <= 999999),
18  INDEX idx_table_time (table_id, started_at),
19  INDEX idx_ended (ended_at)
20 );

```

4.2.10 game_records (战绩)

```

CREATE TABLE game_records (
  id INT PRIMARY KEY AUTO_INCREMENT,
  session_id INT NOT NULL,
  game_id INT NOT NULL,
  winner_player_id INT,
  title_snapshot VARCHAR(100),
  score_json JSON,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (session_id) REFERENCES play_sessions(id) ON DELETE CASCADE,
  FOREIGN KEY (game_id) REFERENCES games(id) ON DELETE RESTRICT,
  FOREIGN KEY (winner_player_id) REFERENCES players(id) ON DELETE SET NULL,
  INDEX idx_session (session_id),
  INDEX idx_game (game_id),
  INDEX idx_winner (winner_player_id)
);

```

4.2.11 player_stats (会员统计)

```

1 CREATE TABLE player_stats (
2   player_id INT PRIMARY KEY,
3   wins INT DEFAULT 0,
4   games INT DEFAULT 0,
5   win_rate VARCHAR(10) DEFAULT '0%',
6   last_win_at DATETIME,
7   updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
8   FOREIGN KEY (player_id) REFERENCES players(id) ON DELETE CASCADE

```

9);

4.3 规范化处理（至 3NF）

4.3.1 第一范式（1NF）

第一范式要求所有属性都是原子的，不能再分解。本系统中：

1. 所有字段都是原子值，不存在重复组
2. 复杂数据（如多人评分）使用 JSON 字段存储，避免创建多行记录
3. 例如 `game_records.score_json` 存储多人评分信息

4.3.2 第二范式（2NF）

第二范式要求消除非主属性对主键的部分依赖。本系统中：

1. 所有表都有单一主键或复合主键
2. 非主属性完全依赖于主键，不存在部分依赖
3. 例如 `reservations` 表中，所有字段都完全依赖于 `id`

4.3.3 第三范式（3NF）

第三范式要求消除非主属性间的传递依赖。本系统中：

1. 大多数表都满足 3NF
2. `game_records.title_snapshot` 是一个受控冗余，用于保存历史快照
3. 这样做的原因是：如果游戏名称被修改，历史战绩仍然能显示当时的游戏名称
4. 这是一个合理的设计权衡，在需求中已明确说明

4.4 完整性约束设计

4.4.1 实体完整性

1. 所有表都有主键（PRIMARY KEY）
2. 主键使用 `AUTO_INCREMENT` 自动生成
3. 主键不允许为空

4.4.2 参照完整性

1. 使用 FOREIGN KEY 约束维护表间关系
2. 删除策略：
 - ON DELETE CASCADE：级联删除（如删除门店时删除其所有桌位）
 - ON DELETE SET NULL：设为空（如删除会员时预约的会员字段设为空）
 - ON DELETE RESTRICT：禁止删除（如删除游戏时若有战绩记录则禁止删除）

4.4.3 用户定义完整性

1. CHECK 约束：

- reserved_end > reserved_start: 预约结束时间必须晚于开始时间
- amount_cents >= 0 AND amount_cents <= 999999: 金额范围限制
- difficulty_level BETWEEN 1 AND 5: 游戏难度限定在 1-5
- seat_capacity BETWEEN 1 AND 20: 桌位容量限定在合理范围

2. ENUM 约束：

- 预约状态包括待入场、已入场、已取消、已完成和未到店，状态只能在业务允许范围内流转

3. UNIQUE 约束：

- venues.name: 门店名称唯一
- game_tables(venue_id, code): 同一门店内桌位编号唯一

4.5 索引设计

为了提升查询性能，设计了以下索引：

表 4.2: 索引设计

表名	索引名	索引列
game_tables	idx_venue_pos	(venue_id, pos_x, pos_y)
game_tables	ix_tables_capacity	(seat_capacity, area_type)
games	ix_games_recommend	(category, difficulty_level, avg_minutes)
reservations	idx_table_time	(table_id, reserved_start, reserved_end)
reservations	idx_status	(status)
play_sessions	idx_table_time	(table_id, started_at)
play_sessions	idx_ended	(ended_at)
game_records	idx_session	(session_id)
game_records	idx_game	(game_id)
game_records	idx_winner	(winner_player_id)

这些索引主要用于：

1. 时间范围查询（预约冲突检测）
2. 状态过滤（查询待处理预约）
3. 关联查询（战绩统计）

第 5 章 系统实现过程

5.1 数据库完整搭建过程

5.1.1 环境准备

1. 安装 MySQL 8.0 或更高版本
2. 配置字符集为 utf8mb4，排序规则为 utf8mb4_unicode_ci
3. 创建数据库用户和权限

5.1.2 初始化脚本执行顺序

系统提供了一套完整的初始化脚本，需要按以下顺序执行：

表 5.1: 数据库初始化脚本

序号	文件名	内容说明
1	01_schema.sql	创建所有表、主键、外键、索引、CHECK 约束、触发器
2	02_procedures.sql	创建存储过程（预约、开台、结算、战绩、统计、智能推荐）
3	03_seed.sql	插入基础种子数据（门店、游戏、桌位、示例会员）
4	04_views.sql	创建视图（排行榜、账单、游戏热度、推荐特征、平面图等）
5	05_bulk_seed.sql	插入大量测试数据（可选，用于压测）
6	06_security_grants.sql	配置用户权限（演示用只读账号、受限应用账号）

5.1.3 执行命令示例

```
1 # 创建数据库
2 mysql -u root -p < db/bootstrap.sql
3
4 # 创建应用用户
5 mysql -u root -p < db/create-app-user.sql
6
7 # 执行初始化脚本
8 mysql -u boardgame -p boardgame < db/init/01_schema.sql
9 mysql -u boardgame -p boardgame < db/init/02_procedures.sql
10 mysql -u boardgame -p boardgame < db/init/03_seed.sql
11 mysql -u boardgame -p boardgame < db/init/04_views.sql
12 mysql -u boardgame -p boardgame < db/init/05_bulk_seed.sql
13 mysql -u boardgame -p boardgame < db/init/06_security_grants.sql
```

Listing 5.1: 数据库初始化命令

5.2 核心数据库对象

5.2.1 触发器

触发器用于维护数据一致性和自动更新聚合数据。系统包含以下触发器：

表 5.2: 系统触发器清单

触发器名称	功能说明
tr_players_after_insert_stats	新增会员时自动创建统计记录
tr_play_sessions_after_insert_state	新增对局时更新桌位状态为占用
tr_play_sessions_after_update_release	对局结束时更新桌位状态为空闲
tr_game_records_after_insert_stats	新增战绩时更新会员统计（胜场、总局数、胜率）

5.2.2 存储过程

存储过程实现了系统的核心业务逻辑：

表 5.3: 系统存储过程清单

存储过程名称	功能说明
sp_reserve_table	创建预约，检测时段冲突和桌位容量是否满足预约人数
sp_checkin_start_session	预约入场，创建对局会话，并记录使用人、电话和人数
sp_start_walkin_session	现场开台，校验桌位容量后创建对局会话，并记录现场使用人信息
sp_end_session_settle	结算关台，更新对局信息
sp_insert_game_record	录入战绩，触发排行榜更新
sp_cancel_reservation	手动取消迟到或未到店预约
sp_expire_overdue_reservations	自动将超出宽限期仍未入场的预约标记为未到店，并释放桌位
sp_report_daily_revenue	日收入统计
sp_report_game_popularity	游戏热度排行
sp_report_table_utilization	桌位利用率分析
sp_recommend_games	根据人数、时长、类型偏好、会员历史和热度推荐桌游
sp_recommend_tables	过滤容量不足和时段冲突桌位，再根据容量、状态和利用率推荐桌位

5.2.3 视图

视图提供了便捷的数据查询接口：

表 5.4: 系统视图清单

视图名称	功能说明
v_table_status_floor	平面图查询，包含桌位状态和当前预约/对局
v_leaderboard	排行榜视图，按胜率排序
v_session_billing_detail	对局账单明细
v_game_catalog_with_stats	游戏目录与热度统计
v_game_recommendation_features	桌游推荐特征视图，聚合目录属性、历史局数和近 30 天热度

5.3 完整性控制实现

5.3.1 实体完整性

1. 所有表都定义了 PRIMARY KEY
2. 使用 AUTO_INCREMENT 自动生成主键值
3. 主键列定义为 NOT NULL

5.3.2 参照完整性

1. 使用 FOREIGN KEY 约束维护表间关系
2. 根据业务需求设置不同的删除策略
3. 例如：删除门店时级联删除其所有桌位

5.3.3 用户定义完整性

1. CHECK 约束：验证时间范围、金额范围等
2. ENUM 约束：限制状态字段的取值
3. UNIQUE 约束：保证某些字段的唯一性
4. NOT NULL 约束：保证必填字段

5.3.4 业务一致性

1. 触发器维护桌位占用/释放状态
2. 触发器维护会员排行聚合数据
3. 存储过程实现原子性的业务操作
4. 例如：预约时同时检测冲突、更新状态、记录日志

5.4 前台功能与后台数据库对象对应

表 5.5: 前台功能与后台数据库对象对应关系

前台功能	后台主要支撑
账号注册登录	app_users、auth_sessions；密码加盐哈希，会话 token 仅保存哈希值
平面图着色与桌位详情	视图 v_table_status_floor + 表 game_table_state、play_sessions、reservations；展示当前使用人和预约队列
预约/取消/未到店处理	存储过程 sp_reserve_table、sp_cancel_reservation、sp_expire_overdue_reservations
入场/现场开台	sp_checkin_start_session、sp_start_walkin_session + 触发器占桌
结算关台	sp_end_session_settle + 触发器释桌
关台后录入战绩	sp_insert_game_record + 触发器更新排行聚合
排行榜/报表	视图 v_leaderboard；过程 sp_report_*
智能推荐	推荐特征视图；桌游推荐过程；桌位调度过程

5.5 智能推荐算法设计与实现

5.5.1 桌游推荐评分模型

系统将桌游推荐设计为可解释的混合评分模型。推荐分数由六类特征加权得到：

```
1 score =  
2   people_score      * 0.25  
3 + duration_score   * 0.15  
4 + category_score   * 0.15  
5 + history_score    * 0.20  
6 + hot_score        * 0.15  
7 + weight_score     * 0.10
```

Listing 5.2: 桌游推荐评分公式

各分项含义如下：

- people_score: 人数是否落在支持范围内
- duration_score: 预计时长与平均时长的接近程度
- category_score: 类型偏好是否匹配
- history_score: 会员历史是否偏好该游戏或同类游戏
- hot_score: 近 30 天战绩记录热度
- weight_score: 门店运营侧人工推荐权重

该算法由 sp_recommend_games 存储过程实现，返回 Top 5 推荐游戏及各分项得分。

5.5.2 桌位调度评分模型

桌位调度算法以“可预约、容量合适、资源均衡”为目标。系统首先过滤占用中桌位和目标时段存在冲突的预约，再计算：

```
1 score =  
2   capacity_score     * 0.55  
3 + availability_score * 0.25  
4 + utilization_score  * 0.20
```

Listing 5.3: 桌位推荐评分公式

其中，capacity_score 避免小队占用大桌或大队挤小桌；availability_score 区分当前空闲与仅在目标时段可用；utilization_score 用于均衡近期桌位使用频率。该算法由 sp_recommend_tables 存储过程实现。

5.6 完成效果

5.6.1 功能完整性

在浏览器中可以完成以下完整的业务流程：

1. 录入预约人数、时间和联系方式，由系统匹配容量合适的空闲桌位
2. 必要时手动选择更高配置桌位或包间
3. 创建预约（客户远程预约通过独立公开页面提交，员工后台负责处理）或现场开台

4. 点开桌位查看桌位档案、当前使用人、联系方式、开台人数和预约队列
5. 对迟到或未到店预约进行手动取消，并由系统定时自动关闭超时预约
6. 在会员详情中查看该会员的预约记录、预约人、人数、联系方式、开始和结束时间
7. 基于人数、时长和会员偏好生成桌游推荐
8. 基于目标时段生成可用桌位推荐
9. 入场开台
10. 计时结算
11. 录入战绩
12. 查看排行榜

5.6.2 数据库完整性

数据库侧可以验证以下对象均已按 SQL 脚本创建：

1. 12 个数据表，覆盖门店、员工档案、账号、登录会话、游戏、桌位、会员、预约、对局、战绩、统计和运行态
2. 4 个触发器
3. 12 个存储过程
4. 5 个视图
5. 多个索引和约束
6. 权限脚本（演示用只读账号、受限应用账号）

5.6.3 测试数据

系统包含以下测试数据：

1. 基础种子数据：1 个门店、25 个游戏、12 个桌位、30 个示例会员
2. 大量测试数据（可选）：约 66 个会员、360+ 条历史对局和战绩记录

这些测试数据可用于功能验证和性能压测。

第 6 章 总结

6.1 与预期目标的符合情况

本课程设计实现了选题预定的 预约—对局—结算—战绩—排行主线，并进一步加入 智能桌游推荐与桌位调度优化，与预期目标 相符。

6.1.1 目标达成情况

1. 需求分析：完整分析了系统的功能需求和非功能需求
2. 概念设计：绘制了 5 个局部 E-R 图和 1 个全局 E-R 图
3. 逻辑设计：设计了 11 个关系模式，满足 3NF 规范
4. 物理实现：编写了完整的 SQL 脚本，包含表、视图、触发器、存储过程
5. 前端实现：实现了完整的用户界面和交互功能
6. 后端实现：实现了 RESTful API 和数据库连接
7. 智能算法：实现了可解释的混合推荐评分模型和桌位调度评分模型

6.2 系统特点

本系统设计具有以下特点：

6.2.1 库内对象齐全

1. 12 个数据表，覆盖所有业务实体、员工档案、后台账号和登录会话
2. 4 个触发器，维护数据一致性
3. 12 个存储过程，实现核心业务逻辑、预约超时处理与智能推荐
4. 5 个视图，提供便捷的数据查询和推荐特征聚合
5. 多个索引，优化查询性能
6. 完整的权限脚本，演示分级权限控制

6.2.2 业务逻辑下沉

1. 将桌位占用/释放状态维护通过触发器实现
2. 将会员排行聚合通过触发器和存储过程实现
3. 将预约冲突检测通过存储过程实现
4. 将超时未到店预约通过存储过程和后端定时任务自动关闭
5. 减轻应用层负担，提升系统可靠性

6.2.3 历史可追溯

1. 通过 `title_snapshot` 字段记录战绩时的游戏名称
2. 避免游戏目录改名破坏历史语义
3. 支持完整的审计追踪

6.2.4 智能化辅助决策

1. 通过 `v_game_recommendation_features` 聚合游戏热度和推荐特征
2. 通过 `sp_recommend_games` 输出桌游 Top 推荐和分项评分
3. 通过 `sp_recommend_tables` 过滤时段冲突并推荐合适桌位
4. 前端展示推荐分数和推荐理由，使系统从记录型管理升级为决策辅助型系统

6.2.5 规范化设计

1. 遵循 3NF 规范，消除数据冗余
2. 在必要处允许受控冗余以提升性能
3. 清晰的表间关系和外键约束

6.2.6 易于扩展

1. 架构清晰，易于添加新的功能模块
2. 存储过程和视图易于修改和扩展
3. 前后端分离，便于独立开发和测试

6.3 存在的问题与有待提高之处

6.3.1 功能层面

1. **预约码与扫码入场**：可在预约成功后生成唯一预约码或二维码，顾客到店扫码后由系统自动核验预约状态、时间窗口和桌位容量，并触发入场开台流程
2. **消息通知**：缺少预约提醒、战绩通知等功能
3. **多人队伍**：目前战绩只支持单人胜者，不支持多人队伍
4. **多局计分**：目前每个对局只能录入一条战绩，不支持多局计分

6.3.2 性能层面

1. **缓存机制**：可以添加 Redis 缓存提升查询性能
2. **查询优化**：某些复杂查询可以进一步优化
3. **批量操作**：可以优化大批量数据导入的性能

6.3.3 安全层面

1. **权限细分**: 目前已实现账号登录认证, 但角色权限仍可继续细分到不同菜单和操作级别
2. **数据加密**: 敏感数据 (如电话号码) 可以加密存储
3. **审计日志**: 可以添加更详细的操作审计日志

6.3.4 运维层面

1. **服务器部署**: 若投入真实运营, 需要配置稳定服务器、域名、HTTPS 和反向代理
2. **环境与存储**: 需要独立管理环境变量, 并使用持久化数据库存储
3. **备份恢复**: 需要建立完整的备份和恢复流程
4. **监警告警**: 需要添加数据库监控和告警机制
5. **版本管理**: 需要建立数据库版本管理和迁移流程

6.4 经验与收获

6.4.1 理论层面

通过本次设计, 深入理解了以下数据库设计理论:

1. **E-R 模型**: 学会了如何用 E-R 图表达复杂的业务关系
2. **关系模式**: 掌握了 E-R 模型向关系模式的转换规则
3. **规范化**: 理解了 1NF、2NF、3NF 的含义和应用
4. **完整性约束**: 学会了如何通过约束保证数据一致性

6.4.2 实践层面

通过本次设计, 获得了以下实践经验:

1. **需求分析**: 学会了如何从业务需求出发进行系统设计
2. **数据库设计**: 掌握了完整的数据库设计流程
3. **SQL 编程**: 学会了编写复杂的 SQL 语句和存储过程
4. **系统集成**: 理解了前后端与数据库的集成方式

6.4.3 工程实践

通过本次设计, 培养了以下工程实践能力:

1. **文档编写**: 学会了编写规范的技术文档
2. **代码规范**: 理解了代码注释和命名规范的重要性
3. **版本管理**: 学会了使用 Git 进行版本控制
4. **系统思维**: 培养了从整体角度思考系统设计的能力

6.4.4 主要收获

1. 认识到 **基数与联系**画清楚后，外键与过程边界自然清晰
2. 体会到 **文档符号规范与 实现命名一致**对团队协作与验收的重要性
3. 理解了 **业务逻辑下沉到数据库层**的优势和权衡
4. 学会了如何在 **规范化与 性能**之间找到平衡点
5. 认识到 **推荐算法**可以与数据库视图、存储过程结合，形成可解释、可演示的智能功能

6.5 改进建议

6.5.1 短期改进

1. 继续细化不同员工角色的菜单权限和操作权限
2. 增加预约提醒、迟到提醒和取消通知
3. 完善错误提示和异常操作审计
4. 进一步优化移动端预约页面

6.5.2 中期改进

1. 支持多人队伍和多局计分
2. 添加消息通知功能
3. 实现数据缓存机制
4. 建立完整的备份恢复流程

6.5.3 长期改进

1. 支持多门店管理
2. 实现高级分析和报表功能
3. 引入大模型解释层，将推荐结果生成更自然的经营建议
4. 建立数据仓库和 BI 系统
5. 支持移动端应用

第 7 章 参考资料

7.1 参考文献

1. MySQL 8.0 Reference Manual. <https://dev.mysql.com/doc/refman/8.0/en/>
2. 王珊, 萨师煊. 《数据库系统概论 (第 5 版)》. 高等教育出版社, 2012.
3. Peter Chen. The Entity-Relationship Model—Toward a Unified View of Data. *ACM Transactions on Database Systems*, 1976.
4. C. J. Date. *An Introduction to Database Systems (8th Edition)*. Addison-Wesley, 2003.
5. 李海翔. 《MySQL 性能优化与架构设计》. 电子工业出版社, 2013.
6. 高性能 MySQL (第 3 版). O'Reilly Media, 2012.

7.2 项目数据库脚本清单

本项目包含以下数据库脚本, 均位于 db/ 目录下:

7.2.1 初始化脚本

1. bootstrap.sql: 创建数据库和初始用户
2. create-app-user.sql: 创建应用用户和权限
3. init/01_schema.sql: 创建所有表、约束、索引、触发器
4. init/02_procedures.sql: 创建存储过程
5. init/03_seed.sql: 插入基础种子数据
6. init/04_views.sql: 创建视图
7. init/05_bulk_seed.sql: 插入大量测试数据 (可选)
8. init/06_security_grants.sql: 配置权限

7.2.2 脚本内容概览

01_schema.sql

该脚本创建以下对象:

- 11 个数据表, 覆盖门店、后台账号、登录会话、游戏目录、桌位状态、会员资料、预约、对局和战绩等核心对象
- 主键和外键约束
- CHECK 约束和 UNIQUE 约束
- 复合索引和单列索引
- 4 个触发器

02_procedures.sql

该脚本创建以下存储过程：

- sp_reserve_table: 创建预约
- sp_checkin_start_session: 预约入场
- sp_start_walkin_session: 现场开台
- sp_end_session_settle: 结算关台
- sp_insert_game_record: 录入战绩
- sp_cancel_reservation: 取消预约
- sp_report_daily_revenue: 日收入统计
- sp_report_game_popularity: 游戏热度排行
- sp_report_table_utilization: 桌位利用率分析
- sp_recommend_games: 智能桌游推荐
- sp_recommend_tables: 智能桌位调度

03_seed.sql

该脚本插入以下基础数据：

- 1 个门店（示例门店）
- 25 个桌游（含类型、难度、平均时长和推荐权重）
- 12 个桌位（3x4 网格）
- 30 个示例会员

04_views.sql

该脚本创建以下视图：

- v_table_status_floor: 平面图查询
- v_leaderboard: 排行榜
- v_session_billing_detail: 账单明细
- v_game_catalog_with_stats: 游戏热度
- v_game_recommendation_features: 推荐特征聚合

05_bulk_seed.sql

该脚本插入大量测试数据（可选）：

- 约 66 个会员
- 约 360 条历史对局记录
- 约 360 条战绩记录

用于性能测试和压测。

06_security_grants.sql

该脚本配置以下权限：

- bg_readonly: 只读账号，用于报表查询
- bg_app: 应用账号，具有 INSERT、UPDATE、DELETE 权限

7.3 相关文档

1. docs/数据库设计说明.md: E-R 图和关系模式的简要说明
2. docs/数据库实施与维护计划.md: 数据库部署、备份、监控等运维指南
3. README.md: 项目总体说明和快速开始指南

7.4 技术栈

7.4.1 数据库

- MySQL 8.0+
- 字符集: utf8mb4
- 排序规则: utf8mb4_unicode_ci

7.4.2 后端

- Node.js 14+
- Express.js 4.x
- mysql2/promise（数据库驱动）
- dotenv（环境变量管理）

7.4.3 前端

- Vite 6.x（构建工具）
- 原生 JavaScript（ES Modules）
- CSS 3（深色主题）

7.4.4 部署

- Docker（容器化）
- Docker Compose（编排）
- Nginx（反向代理）

7.5 相关工具和资源

7.5.1 数据库工具

- MySQL Workbench：数据库设计和管理工具
- DBeaver：通用数据库客户端
- Navicat：数据库管理工具

7.5.2 文档工具

- Mermaid：流程图和 E-R 图绘制
- Graphviz：图表绘制
- LaTeX：文档排版

7.5.3 开发工具

- Visual Studio Code：代码编辑器
- Git：版本控制
- Postman：API 测试

7.6 在线资源

1. MySQL 官方文档：<https://dev.mysql.com/doc/>
2. Mermaid 在线编辑器：<https://mermaid.live>
3. SQL 教程：<https://www.w3schools.com/sql/>
4. 数据库设计最佳实践：https://en.wikipedia.org/wiki/Database_design

致谢

在本次课程设计的资料查阅、需求梳理、数据库脚本联调与文档撰写过程中，得到了多方面的支持和帮助。

首先，感谢 **王文玉** 老师的耐心指导。老师在课程设计的各个阶段都给予了宝贵的建议，特别是在 E-R 模型设计、规范化处理和存储过程编写等方面提供了专业的指导，使本设计能够更加规范和完整。

其次，感谢同学们在环境配置、测试数据验证和系统联调方面的帮助。在数据库初始化、API 测试和前端功能验证过程中，同学们提出了许多宝贵的意见和建议，帮助我发现并修复了多个问题。

再次，感谢学校图书馆和网络资源的支持。通过查阅《数据库系统概论》、MySQL 官方文档等资料，使我能够深入理解数据库设计的理论基础，并将其应用到实践中。

最后，感谢所有为这个项目做出贡献的人。虽然这是一个课程设计项目，但它体现了团队合作的精神，也让我学到了很多实用的技能和知识。

在此，我对所有给予帮助的人表示衷心的感谢。因水平与时间所限，本设计中难免存在疏漏之处，恳请老师和同学们批评指正。

学生签名：李昊阳

日期：2026.5.9

附录 A 数据库 SQL 脚本附录

本附录列出了项目中使用的数据库脚本文件。这些脚本位于 db/ 目录下，可以按顺序执行完成数据库的完整搭建。

A.1 脚本文件清单

表 A.1: 数据库脚本文件清单

序号	文件路径	功能说明
1	db/bootstrap.sql	创建数据库和初始配置
2	db/create-app-user.sql	创建应用用户
3	db/init/01_schema.sql	表、约束、索引、触发器
4	db/init/02_procedures.sql	存储过程
5	db/init/03_seed.sql	基础种子数据
6	db/init/04_views.sql	视图
7	db/init/05_bulk_seed.sql	大量测试数据
8	db/init/06_security_grants.sql	权限脚本

A.2 执行说明

这些脚本需要按顺序执行，原因如下：

1. 01_schema.sql 必须先执行，因为它创建了所有基础表结构
2. 02_procedures.sql 依赖于表结构和触发器
3. 03_seed.sql 依赖于表结构
4. 04_views.sql 依赖于表和种子数据
5. 05_bulk_seed.sql 是可选的，用于测试
6. 06_security_grants.sql 可以最后执行

A.3 脚本特点

本项目的数据库脚本具有以下特点：

1. **可重复执行：**使用 DROP ... IF EXISTS 和 CREATE IF NOT EXISTS 保证脚本可以安全重复执行
2. **完整性高：**包含完整的表结构、约束、索引、触发器、存储过程、视图
3. **注释清晰：**关键逻辑都有详细注释
4. **测试友好：**提供了基础种子数据和大量测试数据
5. **安全性考虑：**包含分级权限脚本

A.4 部署建议

在实际部署时，建议：

1. 开发环境可以执行所有脚本，包括 `05_bulk_seed.sql`
2. 生产环境应跳过 `05_bulk_seed.sql`，避免插入测试数据
3. 生产环境需要修改 `06_security_grants.sql` 中的默认密码
4. 建议使用数据库迁移工具管理后续的结构变更